

RUNNING DBT IN GOOGLE COLAB

Can you set up a full dbt session in Google colab?

Yes

What is dbt – dbt stands for **Data Build Tool**.

dbt Core is a free, open-source command-line tool that lets data teams transform data directly inside their data warehouse using SQL and Jinja. Developed by [dbt Labs](#), it forms the foundation of modern data transformation workflows. The nice thing about dbt is that it runs off of many systems.

You can create projects on Databricks, Fabric and Snowflake for example. Dbt does this by using adapters. These are analogous to the ODBC and JDBC adapters that were used in the past. They allow dbt to interface with different platforms such as Snowflake and Databricks. To do this you need the right adapter.

Dbt also excels at documentation and allows easy documentation of complex jobs.

For this project you will run **dbt-core**, which is open source and free, in Google Colab which is also open source and free.

In addition, you can set up a free student account in Snowflake or Databricks and one of these will be needed for this exercise.

You will need to know your user id, pw, database/catalog, and schema in order to run your queries from dbt.

You will also need some test tables to work with.

These test tables represent the “raw” file data that comes into a data warehouse before any transformation is performed.

Setting Up (**commands are in blue**)

First set up a virtual environment in python called uv.

!pip install uv

Install dbt-core and the dbt-adapter.

For example, if I want to work off of Snowflake I will install the **dbt-snowflake** adapter. If I want to work off of Databricks then I would install the **dbt-databricks** adapter.

For this example I will be using my student Snowflake account.

As stated, you will need to be able to log into your cloud account and be able to create a database and schema for this project.

These represent the default space where your models will be **materialized**.

!uv pip install dbt-core dbt-snowflake

!uv pip install --system dbt-core dbt-snowflake

Check to see if you have installed dbt correctly

!dbt --version

You should see something like the following:

Core:

- installed: 1.12.5
- latest: 1.12.5 - Up to date!

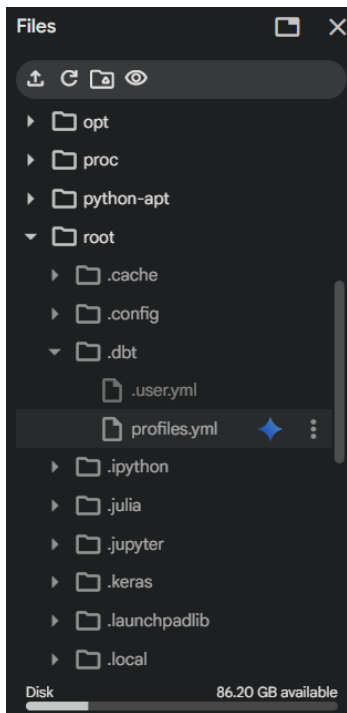
Plugins:

- snowflake: 1.12.1 - Up to date!

Your profile is like a **connection string**. It's in yaml format which is simply as set of key:value pairs and is similar to JSON. It contains information such as the name of your project, the platform you're using, the database and schema, your user name, pw and role. The file is called **profiles.yml** and it's stored in the .dbt directory. In Colab you can find it under the root directory.

#set up the .dbt directory then create your profile
`mkdir -p /root/.dbt/`

Your connection profile will be stored in this folder. It is in .yaml format and it's called **profiles.yml**



As stated, the profiles.yml is simply a connection string. This connection string contains all of the information needed by dbt to make the connection to your account.

Below is an example profiles.yml file setup for Snowflake

```
#Write the profiles.yml File in Python
%%writefile /root/.dbt/profiles.yml
my_snowflake_project:
  target: dev
```

```
outputs:
dev:
  type: snowflake
  account: "xy12345.us-east-1" # Your Snowflake account locator (include region if applicable)
  user: "your_username"
  password: "your_password" # Alternatively, use private_key_path
  role: "your_role"
  warehouse: "your_warehouse"
  database: "your_database"
  schema: "your_schema"
  threads: 4
  client_session_keep_alive: False
```

HOWEVER, you don't want your account info, user name or password to be available in an open source project.

For Colab you will use **SECRETS**. These are inaccessible key/value pairs that can be accessed by dbt when you need to access your database.

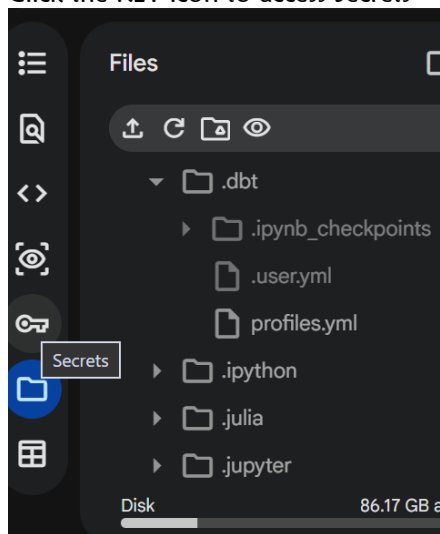
You will use the python code below to generate your profiles.yml on the fly.

Create Secrets in Colab for your account, user name and password.

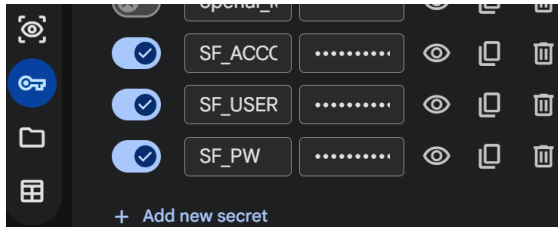
Click the Key icon and enter the information as key value pairs.

Make the information accessible to the notebook by switching the icons so that you see a blue check.

Click the KEY icon to access Secrets



Enter the KEY:VALUE pair for account, user and pw. Use the toggle to make it accessible to the notebook.



Once the information has been entered in Secrets and made accessible to the notebook run the following code.

```
import os
import yaml
from google.colab import userdata
```

Alternative set up - use secrets to generate profile.yml on the fly

1. Fetch credentials securely from Colab Secrets

```
sf_account = userdata.get('SF_ACCOUNT')
```

```
sf_user = userdata.get('SF_USER')
```

```
sf_password = userdata.get('SF_PW')
```

2. Generate a profiles.yml file in the .dbt/ directory.

For example my project name is **dbt_snowflake_in_colab**. Spelling IS important

```
profile_data = {
  "dbt_snowflake_in_colab": {
    "target": "dev",
    "outputs": {
      "dev": {
        "type": "snowflake",
        "account": sf_account,
        "user": sf_user,
        "password": sf_password,
        "role": "accountadmin",      # Non-sensitive config can be hardcoded here
        "warehouse": "compute_wh",
        "database": "dbt_google_colab",
        "schema": "bronze_level_schema",
        "threads": 4,
        "client_session_keep_alive": False
      }
    }
  }
}
```

```
#write out the file to the directory
os.makedirs('/root/.dbt/', exist_ok=True)
with open('/root/.dbt/profiles.yml', 'w') as f:
    yaml.dump(profile_data, f, default_flow_style=False)
```

#Now dbt is ready to use
!dbt --version

Core:
- installed: 1.12.5
- latest: 1.12.5 - Up to date!

Plugins:
- snowflake: 1.12.1 - Up to date!

Your project will be placed under the /content/ directory
Create and navigate to the directory /content/my_project_name.
For example, my project is called **dbt_snowflake_in_colab**. I've put it under the /dbt_parent_folder/

```
#Navigate to your project folder
%cd /content/dbt_parent_folder/
```

Once inside the project folder initialize the dbt project.
It will look for this project name in the profiles.yml file.
!dbt init dbt_snowflake_colab --skip-profile-setup

Your dbt project has been created and you are ready to begin.
If you see errors then check for spelling and also that you navigated into the correct directory.

```
18:12:15 Running with dbt=1.12.5
18:12:15 Your new dbt project "dbt_snowflake_colab" was created!
Initialized new project in /content/dbt_snowflake_in_colab/dbt_snowflake_colab/dbt_snowflake_colab
```

For more information on how to configure the profiles.yml file,
please consult the dbt documentation here:

<https://docs.getdbt.com/docs/configure-your-profile>

One more thing:

Need help? Don't hesitate to reach out to us via GitHub issues or on Slack:

<https://community.getdbt.com/>

Happy modeling!

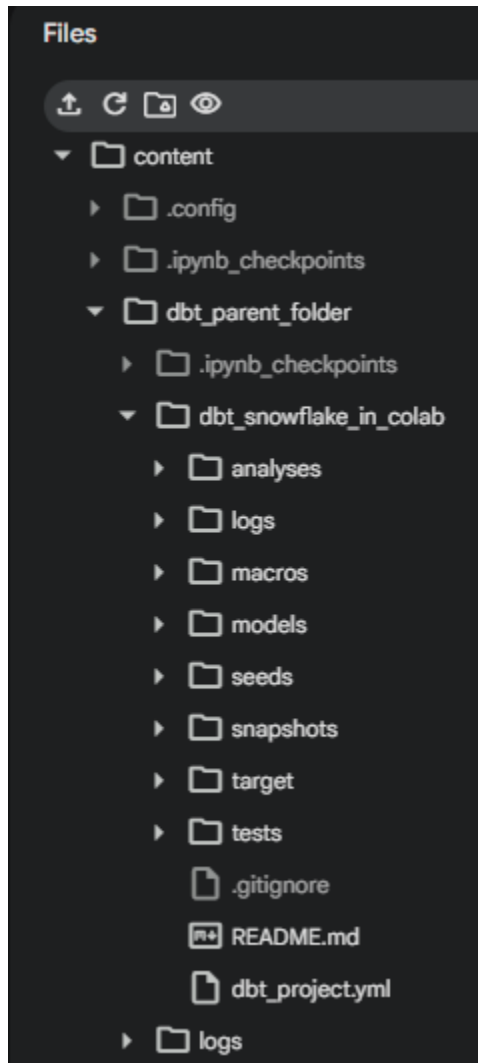
To make sure that you are in the correct directory before running a test project

`%ls`

You should see the work folders listed below.

In order to run a project dbt has to be able to access the `dbt_project.yml` file so you need to be in the correct directory.

For me this is `/content/dbt_parent_folder/dbt_snowflake_in_colab/`



Run your first project!

There are two example models in the .dbt set up that you can create.

After a successful run these will be located in your platform account under your database and schema.
This is done to verify that everything is working.

!dbt run

It may take several tries but you will be successful!

/content/dbt_parent_folder/dbt_snowflake_in_colab

19:19:50 Running with dbt=1.12.5

19:19:51 Registered adapter: snowflake=1.12.1

19:19:51 Unable to do partial parsing because profile has changed

19:19:54 Found 2 models, 4 data tests, 562 macros

19:19:54

19:19:54 Concurrency: 4 threads (target='dev')

19:19:54

19:19:56 1 of 2 START sql table model bronze_level_schema.my_first_dbt_model [RUN]

19:19:57 1 of 2 OK created sql table model bronze_level_schema.my_first_dbt_model [SUCCESS 2 in 1.37s]

19:19:57 2 of 2 START sql view model bronze_level_schema.my_second_dbt_model [RUN]

19:19:58 2 of 2 OK created sql view model bronze_level_schema.my_second_dbt_model [SUCCESS 1 in 1.25s]

19:19:59

19:19:59 Finished running 1 table model, 1 view model in 0 hours 0 minutes and 4.54 seconds (4.54s).

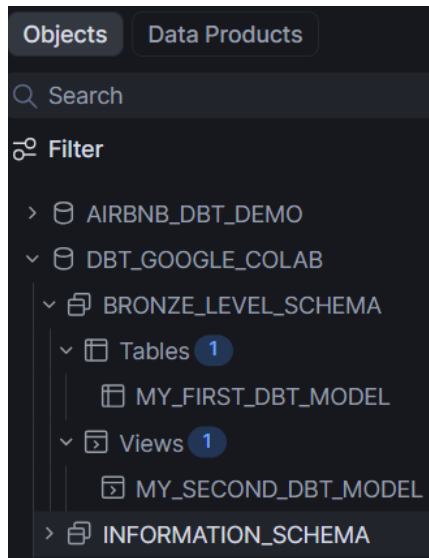
19:19:59

19:19:59 Completed successfully

19:19:59

19:19:59 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=2

Log in to your platform. For this exercise I used Snowflake but the results are the same in Databricks
View your **new models** in your assigned database and schema.



Happy modeling!